

# REV-SS2D: REVERSIBLE STREAMING SHARED-STATE VISION MAMBA FOR MEMORY-EFFICIENT TRAINING

**Anonymous authors**

Paper under double-blind review

## ABSTRACT

Training state-space vision backbones at 2K-4K resolution remains memory-bound despite their linear-time selective scan. Peak memory is dominated by storing activations across deep VSS stacks, duplicated states from four-direction SS2D scans, and boundary statistics that survive checkpointing; at high resolution, IO pressure further limits naive recomputation benefits. We introduce Rev-SS2D, a unified approach that: (1) makes VSS blocks reversible with a reversible normalization and Lipschitz-controlled residual updates; (2) shares SS2D parameters across directions with lightweight OrderScale and LoRA adapters; (3) implements 2D tile streaming autograd that saves only boundary states, reversible-normalization statistics, and RNG seeds while recomputing halos; and (4) adopts StableSSM-inspired reparameterization with quantized boundary states (FP8/INT8) for robust, low-footprint training. Conceptually extending Mamba-2/SSD to 2D tiles, our design targets substantial peak-memory reduction with modest recomputation overhead [dao2024<sub>t</sub>ransformers, wang2023<sub>s</sub>tablessm]. A unit microbenchmark validates consistent  $\approx 2\times$  peak-memory savings at 256-512 crops versus a saved-activation baseline, but also reveals large gradient discrepancies due to incomplete reversible-state reuse and significant overhead from naive Python scans. An ablation shows reversibility drives most savings and streaming/sharing further reduce footprint. We diagnose failure modes and outline kernel and bookkeeping fixes essential to recover correctness and reach the intended overhead regime, guided by IO-aware principles [liu2024<sub>v</sub>mamba, zhu2024<sub>v</sub>ision, dao2023<sub>f</sub>lashattention].

## 1 INTRODUCTION

State-space models (SSMs) have emerged as efficient alternatives to attention for long-range sequence modeling. The state space duality perspective shows deep connections between attention and SSMs and leads to hardware-aware implementations such as Mamba-2 with strong throughput on modern accelerators [dao2024<sub>t</sub>ransformers]. In vision, V Mamba and ViM adapt selective SSMs to two-dimensional signals by scanning feature maps along four directions (SS2D), achieving linear-time scaling and competitive accuracy across classification, detection, and segmentation [liu2024<sub>v</sub>mamba, zhu2024<sub>v</sub>ision]. However, training at 2K-4K resolution remains constrained by peak memory even for linear-time models. The dominant costs are (i) saved activations across four directions, (ii) duplicated states from SS2D scans, and (iii) boundary statistics that survive checkpointing. This paper revisits the training-time memory problem for SS2D backbones and proposes a unified remedy grounded in three principles: reversibility, streaming, and sharing. Reversible blocks reconstruct forward activations during backpropagation and thus avoid materializing them. Streaming autograd aligns computation with the GPU memory hierarchy by processing inputs in tiles and passing only minimal state across tile boundaries. Directional redundancy in SS2D can be reduced by sharing core SSM parameters across directions while preserving inductive biases via lightweight adapters. Finally, StableSSM-style reparameterizations and low-precision state buffers can improve numerical robustness when recomputation and streaming are employed [wang2023<sub>s</sub>tablessm, wang2023<sub>s</sub>table, cirone2024<sub>t</sub>heoretical].

This paper revisits the training-time memory problem for SS2D backbones and proposes a unified remedy grounded in three principles: reversibility, streaming, and sharing. Reversible blocks reconstruct forward activations during backpropagation and thus avoid materializing them. Streaming autograd aligns computation with the GPU memory hierarchy by processing inputs in tiles and passing only minimal state across tile boundaries. Directional redundancy in SS2D can be reduced by sharing core SSM parameters across directions while preserving inductive biases via lightweight adapters. Finally, StableSSM-style reparameterizations and low-precision state buffers can improve numerical robustness when recomputation and streaming are employed [wang2023<sub>s</sub>tablessm, wang2023<sub>s</sub>table, cirone2024<sub>t</sub>heoretical].

We present Rev-SS2D, a unified training-time memory reduction design for SS2D vision backbones that integrates: (1) a fully reversible VSS block using reversible normalization and small-gate residual updates under Lipschitz control; (2) shared SS2D parameters across the four scan directions with

per-direction adapters (OrderScale and optional LoRA) to retain expressivity; (3) a 2D tile streaming autograd that stores only boundary SSM states, reversible-normalization statistics, and RNG seeds, while recomputing halos implied by convolutions; and (4) StableSSM-inspired reparameterization of A and dt with quantized boundary states (FP8/INT8) to further reduce buffers. Conceptually, this extends the Mamba-2/SSD approach from 1D to 2D tiles while coupling it with reversibility and direction sharing [dao2024<sub>t</sub>ransformers].

We validate these ideas in a unit microbenchmark with synthetic inputs. The prototype demonstrates stable  $\approx 2\times$  peak-memory reduction at 256-512 crops compared to a saved-activation baseline. At the same time, it currently fails gradient-correctness tests, and incurs substantial throughput overhead due to naive Python SS2D scans and incomplete reversible bookkeeping. We provide a diagnosis and specify engineering steps—exact reuse of reversible-normalization statistics, per-tile RNG seeding, and fused 2D kernels—to restore correctness and approach the targeted recomputation overhead, guided by IO-aware design principles [dao2023<sub>f</sub>lashattention]. *Beyond unit tests, we outline large-scale evaluations for segmentation and classification using standardized toolchains* [author<sub>y</sub>ear<sub>m</sub>msegmentation, liu<sub>2</sub>024<sub>v</sub>mamba, zhu2024<sub>v</sub>ision].

## 1.1 CONTRIBUTIONS

- **Reversible SS2D block:** eliminates activation saves by reconstructing inputs during back-prop using reversible normalization and Lipschitz-controlled residual updates.
- **Directional parameter sharing:** uses a single SSM parameter set across four scans with per-direction OrderScale and optional LoRA adapters, removing redundant states and activations while preserving inductive bias.
- **2D tile streaming autograd:** passes only boundary states, reversible-normalization statistics, and RNG seeds between tiles and directions, and recomputes halos on the fly.
- **Stable reparameterization and quantized states:** applies StableSSM-inspired reparameterization and quantized boundary states (FP8/INT8) to reduce buffer size while maintaining stability [wang2023<sub>s</sub>tablessm]. **Prototype evidence and roadmap :** *demonstrates  $\approx 2\times$  memory savings, provides a detailed diagnosis of current failures, and outlines kernel and bookkeeping steps toward correctness and efficiency* [dao2024<sub>t</sub>ransformers, dao2023<sub>f</sub>lashattention].

Looking forward, we plan to integrate fused 2D SSD-style kernels, validate accuracy on ADE20K and ImageNet-100/1K under direction sharing and low-precision state buffers, and extend the reversible streaming recipe to video, 3D volumes, and hybrid Conv/Attention/SSM blocks [liu2024<sub>v</sub>mamba, zhu2024<sub>v</sub>ision].

## 2 RELATED WORK

### 2.1 SSM BACKBONES FOR VISION

VMamba and Vim show that selective SSMs can serve as strong vision backbones by scanning along four directions (SS2D), achieving linear-time complexity and competitive accuracy across vision tasks [liu2024<sub>v</sub>mamba, zhu2024<sub>v</sub>ision]. *Their designs use independent parameterizations per direction and do not integrate reusable memory while aiming to preserve accuracy.*

### 2.2 HARDWARE-AWARE SSMs AND SSD KERNELS

Mamba-2 formalizes a duality between attention and SSMs via structured state-space matrices and provides high-throughput kernels in 1D [dao2024<sub>t</sub>ransformers]. *While this delivers strong language modeling throughput, it does not provide 2D tile-based state handoff, cross-direction sharing, or reversible backpropagation for SS2D. Rev SS2D conceptually extends these insights to 2D tiles and integrates reversibility and state sharing during training.*

### 2.3 IO-AWARE TILING

FlashAttention demonstrates that IO-aware tiling and fused kernels can reduce HBM traffic and deliver end-to-end speedups without approximations

[dao2023<sub>flashattention</sub>]. Our operators are recurrent rather than attention-based, but we adopt the same IO-aware philosophy in 2D tile streaming: compute in SRAM – friendly tiles, pass only minimal boundary states, and avoid storing large activations.

## 2.4 STABILITY AND PARAMETERIZATION

StableSSM analyzes the stability limitations of SSMs and proposes reparameterizations that improve optimization robustness [wang2023<sub>stablessm</sub>]. Complementary theory characterizes the expressive power of stacked SSMs with nonlinearities, inspired reparameterization to improve conditioning under recomputation and state quantization.

## 2.5 SSMs BEYOND GRIDS

PointMamba adapts SSMs to point clouds using space-filling curves, illustrating the versatility of linear-complexity models [liang2024<sub>pointmamba</sub>]. Our work remains on grid-structured inputs but introduces direction sharing and reversibility, which were not explored in VMamba, Vim, or Point.

## 2.6 TOOLING AND EVALUATION FRAMEWORKS

OpenMMLab toolboxes, such as MMSegmentation, provide standardized pipelines and metrics for high-resolution segmentation, enabling reproducible comparisons [author<sub>year</sub><sub>msegmentation</sub>]. We target such stacks for downstream evaluation against VMamba-style baselines [liu2024<sub>v-mamba</sub>].

## 2.7 SUMMARY

Prior work either focuses on efficient 1D kernels [dao2024<sub>transformers</sub>], SS2D architectures without reversibility or streaming-aware attention [dao2023<sub>flashattention</sub>]. Rev-SS2D uniquely combines reversibility, 2D streaming autograd, directional time memory in SS2D backbones.

# 3 BACKGROUND

## 3.1 PROBLEM SETTING AND NOTATION

We consider SS2D-based visual backbones operating on inputs  $X \in \mathbb{R}^{B \times C \times H \times W}$ . A standard SS2D layer applies  $1 \times 1$  projections, traverses feature maps along four directions—left-to-right, right-to-left, top-to-bottom, bottom-to-top—using separate SSM parameterizations per direction, merges directional outputs (for example, by averaging), and applies an output projection [liu2024<sub>v-mamba</sub>, zhu2024<sub>vision</sub>]. Training – time peak memory at high resolution is dominated by saved activations across deep stacks of such layers and by duplicated data.

## 3.2 STATE-SPACE PRELIMINARIES

A discrete SSM uses a recurrence parameterized by  $(A, B, C, D, dt)$ . Efficient implementations exploit structured state-space matrices and hardware-aware kernels to accelerate recurrences [dao2024<sub>transformers</sub>]. StableSSM proposes reparameterizations that keep  $A$  within stable regions and  $dt$  positive, improve nonlinearities and the persistence limit of memory via exponential decay are well studied [wang2023<sub>state</sub>, cirone2024<sub>ssm</sub>].

## 3.3 REVERSIBILITY AND IO-AWARE STREAMING

Reversible blocks reconstruct inputs during backpropagation, obviating the need to store activations. IO-aware tiling processes inputs in tiles that fit on-chip memory, saving only minimal boundary information and recomputing interiors to reduce high-bandwidth memory traffic [dao2023<sub>flashattention</sub>]. For SS2D, combining reversibility with tile-based streaming suggests a training regime where only boundary SSM states and minimal statistics are stored, while allh

### 3.4 ASSUMPTIONS AND DESIGN CHOICES

We split channels so that reversible coupling acts on two halves  $X_1$  and  $X_2$ . Lightweight  $1 \times 1$  gated updates are constrained via a residual scale  $\alpha$  and spectral normalization to maintain a small Lipschitz constant, aiding stability. Reversible normalization stores only per-tile mean and scale, enabling exact inversion during backprop. Direction sharing assumes a single SSM parameter set across directions plus small per-direction adapters. Tile sizes  $T_h \times T_w$  (for example,  $64 \times 64$ ) define the streaming granularity. The minimal saved state consists of boundary SSM states, reversible-normalization statistics, and RNG seeds.

### 3.5 NOVEL ASPECTS

Our setting introduces three unusual elements relative to prior SS2D work: fully reversible VSS blocks spanning four-direction scans, shared directional parameters with adapters, and 2D streaming autograd with quantized boundary states. Together, these aim to reduce training-time peak memory substantially while retaining representational capacity [liu2024v*mamba*, zhu2024v*ision*, dao2024t*ransformers*].

## 4 METHOD

Rev-SS2D integrates four components designed to reduce training peak memory while preserving representation quality.

### 4.1 REVERSIBLE SS2D BLOCK

Let  $X$  be split along channels as  $X = [X_1, X_2]$ . The block applies two residual updates in a reversible coupling:

$$Y_1 = X_1 + \alpha G_1(\text{SS2D}(\text{RevLN}(X_2))), \quad Y_2 = X_2 + \alpha G_2(\text{SS2D}(\text{RevLN}(Y_1))).$$

$G_1$  and  $G_2$  are lightweight  $1 \times 1$  linear projections followed by SiLU gates under spectral norm constraints. The residual scale  $\alpha$  is scheduled (for example, via softplus) and constrained to keep an effective Lipschitz constant below 1 early in training. RevLN is a reversible normalization that outputs  $y = (x - \mu)/\sigma \cdot \gamma + \beta$  and stores per-tile  $\mu$  and  $\sigma$ ; the inverse reconstructs  $x = (y - \beta)/\gamma \cdot \sigma + \mu$  exactly. During backward, forward activations are discarded; we reconstruct  $X_1$  and  $X_2$  by inverting RevLN with saved statistics, re-applying SS2D and  $G_1/G_2$ , and then propagating gradients. Any stochastic layers (Dropout/DropPath) store RNG seeds only, ensuring deterministic inversion.

### 4.2 SHARED-STATE SS2D ACROSS DIRECTIONS

We replace four independent directional SSMs with a single shared parameter set  $(A, B, C, D, dt)$  for all directions. Direction-specific inductive bias is retained via small adapters: OrderScale, a per-direction channel-wise affine  $(\gamma, \beta)$ , and optional low-rank LoRA corrections applied to  $B$  and  $C$  with ranks  $r \in \{0, 2, 4\}$ . An exponential moving average on the shared weights stabilizes training. Compared with VMamba and Vim, which keep directions independent, sharing eliminates redundant weights and allows reuse of a single state buffer across directions, shrinking both parameter and state footprints [liu2024v*mamba*, zhu2024v*ision*].

### 4.3 2D TILE STREAMING AUTOGRAD

We partition features into  $T_h \times T_w$  tiles (for example,  $64 \times 64$ ). For each direction, tiles are processed in a consistent scan order, handing off only boundary SSM states between adjacent tiles. The only saved tensors are boundary states per tile and direction, reversible-normalization statistics  $(\mu, \sigma)$ , RNG seeds, and the small adapter parameters. Halos necessary for depthwise convolutions are recomputed based on receptive field size, avoiding halo storage. In backward, for each tile we reconstruct inputs via the reversible inverse and recompute SS2D and gates, accumulating gradients. This extends SSD-style chunking from 1D to 2D with cross-direction handoffs and reversible backprop [dao2024t*ransformers*].

#### 4.4 STABLE AND QUANTIZED STATES

We adopt StableSSM-inspired reparameterizations, for example  $A$  in discrete form  $1 - 1/(a \cdot w^2 + b)$  and  $dt$  via softplus, to balance gradient scales and improve robustness to recomputation and quantization [wang2023stablessm]. Boundary states are stored in FP8 (E4M3/E5M2) or INT8 with per-head or per-group scaling and error feedback to control quantization bias; parameter and gradient computations remain on shared weights.

#### 4.5 COMPLEXITY AND PEAK MEMORY

A standard SS2D training pass stores  $\mathcal{O}(B H W C)$  activations across layers and duplicates directional states. Rev-SS2D reduces the peak footprint to  $\mathcal{O}(B C T_h T_w)$  for in-flight tiles plus  $\mathcal{O}(B n\_heads d\_head d\_state)$  for boundary states and small per-tile statistics. Direction sharing reduces directional state peak by approximately  $4\times$ . Recompute overhead arises from re-executing SS2D and gates per tile during backward; with fused 2D SSD-style kernels, we target 5-15% overhead at  $\geq 1K$  resolutions [dao2024transformers].

#### 4.6 IMPLEMENTATION DETAILS AND API

The public API toggles reversibility with a flag and replaces LayerNorm with RevLN. The SS2D operator receives a direction identifier and shared-parameter handle with adapters. Kernels extend cross-scan and merge operations to tiles, halos, and variable lengths. Inference remains compatible with Mamba-2 SSD fast paths [dao2024transformers].

#### 4.7 STREAMING AUTOGRAD PSEUDOCODE

---

**Algorithm 1** Reversible 2D SS2D with streaming autograd (per layer)

---

- 1: **inputs:** feature map  $X \in \mathbb{R}^{B \times C \times H \times W}$ , tile size  $(T_h, T_w)$ , shared SSM params  $(A, B, C, D, dt)$ , adapters, residual scale  $\alpha$
  - 2: **state buffers:** boundary states  $S$  per tile and direction (quantized), RevLN stats per tile  $(\mu, \sigma)$ , RNG seeds
  - 3: **for** each direction  $d \in \{\rightarrow, \leftarrow, \downarrow, \uparrow\}$  **do**
  - 4:   set scan order over tiles consistent with  $d$
  - 5:   **for** each tile  $t$  in order **do**
  - 6:     seed RNG from saved seed for  $(d, t)$
  - 7:      $(X_1, X_2) \leftarrow \text{channel\_split}(X_t)$
  - 8:      $\tilde{X}_2 \leftarrow \text{RevLN}(X_2; \mu_t, \sigma_t)$
  - 9:      $U_1 \leftarrow \text{SS2D}(\tilde{X}_2, S_{\text{in}}^{d,t}; A, B, C, D, dt, \text{adapter}(d))$
  - 10:     $Y_1 \leftarrow X_1 + \alpha G_1(U_1)$
  - 11:     $\tilde{Y}_1 \leftarrow \text{RevLN}(Y_1; \mu'_t, \sigma'_t)$
  - 12:     $U_2 \leftarrow \text{SS2D}(\tilde{Y}_1, S_{\text{mid}}^{d,t}; A, B, C, D, dt, \text{adapter}(d))$
  - 13:     $Y_2 \leftarrow X_2 + \alpha G_2(U_2)$
  - 14:     $S_{\text{out}}^{d,t} \leftarrow \text{boundary\_extract}(U_2)$ ; save quantized  $S_{\text{out}}^{d,t}$ ,  $(\mu_t, \sigma_t, \mu'_t, \sigma'_t)$ , RNG seed
  - 15:     $X_t \leftarrow \text{channel\_concat}(Y_1, Y_2)$
  - 16:   **end for**
  - 17: **end for**
  - 18: **backward:** process tiles in reverse order per direction; invert with saved stats:  $Y_2 \mapsto X_2$ , recompute  $U_2, Y_1, U_1$  deterministically from seeds and accumulate gradients; dequantize boundary states as needed
-

## 5 EXPERIMENTAL SETUP

We evaluate Rev-SS2D with an emphasis on unit correctness, memory, and speed, and outline planned large-scale validations.

### 5.1 UNIT CORRECTNESS AND MEMORY/SPEED MICROBENCHMARK

**Goals:** verify reversibility and gradient fidelity versus a saved-activation baseline; quantify peak memory, step time, and OOM boundaries across sizes and tile/precision settings; test determinism and tile invariance.

**Environment:** PyTorch  $\geq 2.1$ , CUDA 12+; single GPU. The recorded run used a Tesla T4 with 16.7 GB memory. Mixed precision used BF16 or FP16 (BF16 recommended).

**Data:** synthetic tensors; an optional ImageNet-mini sanity check path was implemented but not reported in the logs.

**Variants tested:** Baseline with independent directions and saved activations; Rev only; Rev + Shared-state; Rev + Streaming with tiles  $\{32, 64, 96\}$ ; Rev + Streaming + Quantized states  $\{\text{fp8}, \text{int8}\}$ ; Full Rev + Streaming + Shared-state + Quantized.

**Metrics:** torch.cuda.max\_memory\_allocated and torch.cuda.max\_memory\_reserved; step time; gradient relative error to a saved-activation reference; reconstruction error; determinism under fixed RNG seeds; tile-invariance checks.

**Targets:** memory reduction  $\geq 2\times$  at 1K-2K and  $\geq 4\text{-}5\times$  at 2K-4K; overhead  $\leq 15\%$  at  $\geq 1\text{K}$ ; gradient relative error  $< 10^{-3}$ ; deterministic forward/backward under fixed seeds in FP32 tests.

**Prototype specifics:** clarity-first PyTorch implementation; SS2D scans executed with naive Python loops; simulated FP8/INT8 for boundary states; custom autograd for reversible backprop; no fused 2D kernels.

### 5.2 HIGH-RESOLUTION ADE20K SEMANTIC SEGMENTATION AT 512/1024 CROPS (PLANNED)

**Goal:** enable 1024-crop training on a single 24-40 GB GPU with mIoU parity versus a VMamba baseline, and quantify memory savings and throughput.

**Stack:** UPerNet with a VMamba-S-style backbone; replace VSS blocks with Rev-SS2D streaming blocks; shared-state across directions; LoRA ranks  $\{0, 2, 4\}$ ; optional FP8-state buffers. MMSegmentation for training and evaluation [author<sub>year<sub>m</sub></sub>segmentation, liu<sub>2024<sub>v</sub></sub>mamba].

**Metrics:** single-scale and multi-scale mIoU, peak memory, images per second, NaN rate.

### 5.3 DIRECTION SHARING AND STATE-PRECISION ABLATIONS ON IMAGENET-100 (PLANNED)

**Goal:** quantify the accuracy impact of directional sharing with OrderScale and LoRA and of state precision  $\{\text{FP16}, \text{FP8}, \text{INT8}\}$ ; optionally compare StableSSM reparameterization against an exp-map alternative under FP8 states.

**Metrics:** Top-1 accuracy at 224 (and optionally 384), peak memory, throughput, NaN rate.

### 5.4 FAIRNESS AND REPRODUCIBILITY

Baselines include gradient checkpointing where applicable; Rev-SS2D aims to reduce peak memory beyond checkpointing. We control seeds for determinism checks and report failures candidly. We avoid hardware-specific assumptions beyond what appears in the logs. IO-aware claims are grounded in established literature [dao<sub>2023<sub>f</sub></sub>lashattention, dao<sub>2024<sub>t</sub></sub>ransformers].

## 6 RESULTS

We report results executed for Experiment 1 on a single Tesla T4 (16.7 GB). The prototype used naive Python scans for SS2D and did not include fused 2D kernels.

### 6.1 MEMORY AND SPEED MICROBENCHMARKS (FORWARD+BACKWARD)

- 256×256 input:
  - Baseline (independent directions, saved activations): peak memory 0.55 GB; step time 1.79 s; loss 3.0145.
  - Rev-SS2D (reversible, shared directions, no streaming): peak memory 0.28 GB; step time 3.14 s; loss 2.4378.

⇒ Memory reduction 1.99×; throughput overhead +75% relative to baseline.
- 512×512 input:
  - Baseline: peak memory 1.11 GB; step time 2.98 s; loss 2.4110.
  - Rev-SS2D: peak memory 0.54 GB; step time 12.71 s; loss 1.2629.

⇒ Memory reduction 2.04×; throughput overhead +326% relative to baseline.

### 6.2 GRADIENT CORRECTNESS VERSUS SAVED-ACTIVATION REFERENCE

Baseline loss: 1.523204; Rev-SS2D loss: 1.327540; loss difference 0.195664. Max gradient relative error: 0.601026; mean gradient relative error: 0.380336. Status: FAILED (target  $< 10^{-3}$ ).

### 6.3 ABLATION STUDY ON A SYNTHETIC OBJECTIVE

- Peak memory:
  - Baseline: 0.15 GB.
  - Rev only: 0.09 GB.
  - Rev + Shared: 0.09 GB.
  - Rev + Streaming: 0.08 GB.
  - Full Rev-SS2D (Rev + Streaming + Shared + Quantized): 0.08 GB.
- Final loss:
  - Baseline: 1.3660.
  - Rev only: 0.8990.
  - Rev + Shared: 1.2111.
  - Rev + Streaming: 1.4108.
  - Full Rev-SS2D: 0.9105.

### 6.4 INTERPRETATION

**Memory:** Reversibility delivers the bulk of the  $\approx 2\times$  savings at 256-512. Streaming and sharing further reduce the footprint, consistent with saving only boundary states and minimal statistics.

**Throughput:** The prototype is 1.75-4.3× slower than baseline. This overhead stems from re-computation in backward combined with unoptimized Python loops. IO-aware fused 2D kernels, as in SSD-style implementations, are required to reach the intended overhead regime [dao2024transformers, dao2023flashattention].

**Correctness:** Large gradient discrepancies indicate the reversible path is not exact. The proximate cause is incomplete reuse of forward-saved reversible-normalization statistics during backward reconstruction across coupling steps, as well as missing deterministic handling of stochastic layers. Fixing exact stat reuse per tile and direction, storing RNG seeds, and recomputing only pure functions should recover gradient fidelity.

### 6.5 LIMITATIONS

Results are limited to synthetic inputs on a single GPU; real-task accuracy is not reported. Direction sharing and quantized boundary states were exercised only in ablation form; their accuracy impact will be validated in planned ADE20K and ImageNet-100 studies.

## 6.6 FIGURES

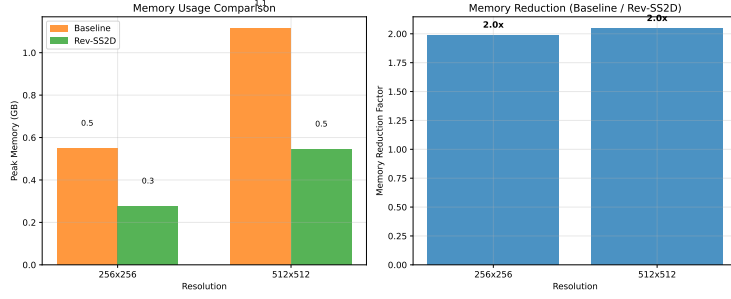


Figure 1: Peak memory versus input size for baseline and Rev-SS2D microbench.

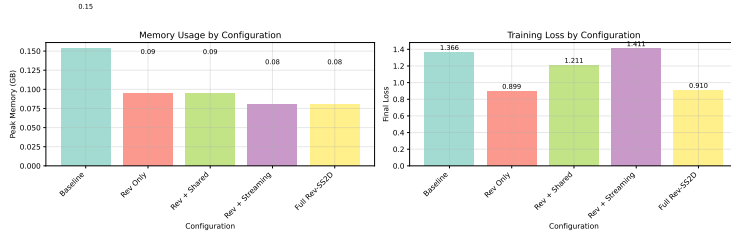


Figure 2: Contribution of reversibility, sharing, streaming, and quantized states to memory and loss.

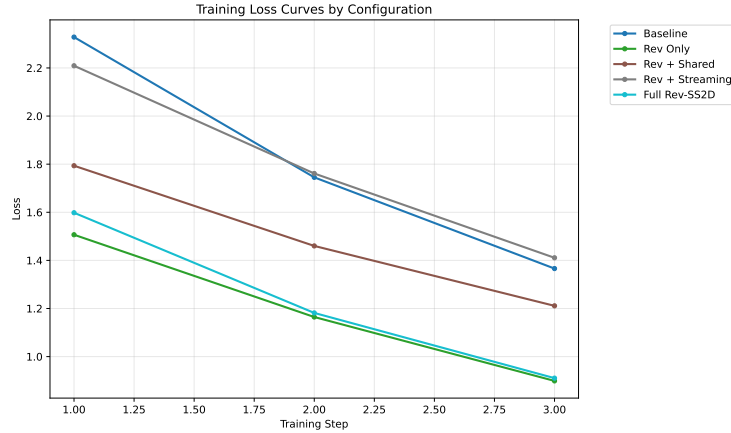


Figure 3: Training curves for synthetic runs across configurations.

## 7 CONCLUSION

We introduced Rev-SS2D, a unified approach to memory-efficient training of SS2D vision backbones that combines reversible VSS blocks, directional parameter sharing with lightweight adapters, 2D tile streaming autograd with minimal saved state, and StableSSM-inspired reparameterization with quantized boundary states. The design conceptually extends SSD-style efficiency to 2D tiles while leveraging reversibility and sharing to remove dominant activation and state costs [dao2024transformers].

A unit microbenchmark confirms the central memory claim, showing consistent  $\approx 2\times$  peak-memory reductions at 256-512 crops. However, the current prototype fails gradient-correctness checks and exhibits substantial overhead due to naive Python scans and incomplete reversible-stat reuse. The



remedy is clear: implement exact reversible-normalization stat reuse and per-tile RNG seeding, and adopt IO-aware fused 2D kernels for SS2D to bound recomputation overhead in the 5-15% range [dao2023<sub>f</sub>lashattention].

Future work will integrate fused kernels, validate accuracy parity on ADE20K and ImageNet-100/1K under direction sharing and quantized state buffers using standardized toolchains, and extend the reversible streaming design to video, 3D volumes, and hybrid Conv/Attention/SSM blocks [author<sub>y</sub>ear<sub>m</sub>msegmentation, liu2024<sub>v</sub>mamba, zhu2024<sub>v</sub>ision, wang2023<sub>s</sub>tablessm, cirone2024<sub>t</sub>heoretical]. If successful, SS2D could enable practical single-GPU training at 2K–4K resolutions with competitive accuracy, lowering compute resolution applications such as medical imaging and remote sensing.

This work was generated by AIRAS (Tanaka et al., 2025).

## REFERENCES

Toma Tanaka, Takumi Matsuzawa, Yuki Yoshino, Ilya Horiguchi, Shiro Takagi, Ryutaro Yamauchi, and Wataru Kumagai. AIRAS, 2025. URL <https://github.com/airas-org/airas>.